

NAVIGATION

Table of Contents

01 · Technology Stack

Frontend, backend, data, infra & AI — every choice with rationale

02 · System Architecture

End-to-end: client → CDN → gateway → 18 services → data → external

03 · Repository Structure

Full monorepo: apps, 18 services, packages, infra (CloudArc base)

04 · Microservices Design

All 18 services — runtime, compute, responsibilities, CSR pattern

05 · Micro-Frontend Architecture

Module Federation, 12 MFEs, shared singletons, PRD module coverage

06 · Database Schema

All tables across 13 PRD modules — fields, types, indexes, relations

07 · GST Integration

E-Invoice, E-way Bill, IRN flow, ASP/GSP, retry & CGST/IGST logic

08 · WhatsApp Integration

Meta Cloud API, inbound/outbound flows, 11 templates, 24h window

09 · Warehouse & Serial Tracking

Pallet hierarchy, GRN flow, barcode scanning, stock formula

10 · Mobile App Architecture

React Native, offline-first, GraphQL gateway, QR scanning

11 · Event-Driven Architecture

10 SQS queues, SNS topics, EventBridge rules, DLQ strategy

12 · Security Architecture

Zero-trust 5-layer, RLS, JWT, WAF, PII encryption, RBAC matrix

13 · CI/CD Pipeline

GitHub Actions, Turborepo affected, per-service deploy

14 · Module Deployment Map

Every service mapped to AWS compute, MFE, and DB tables

15 · CDK Stack Structure**16 · Environment Setup**

18/05/2026, 11:32	SolarOS – Technical Initiation Document v1.0
7 stacks, L3 constructs, dependency chain, CloudArc extensions	docker-compose, env variables, prerequisites, local dev commands
17 · API Contract Conventions Envelope, pagination, versioning, error codes, optimistic locking	18 · Day 1 Engineering Checklist Ordered tasks from repo creation to first API call on staging

CHAPTER 01

Technology Stack — Decisions & Rationale

Every technology choice has a concrete rationale tied to the SolarOS workload. No cargo-culting — each tool is the best fit for this problem at this team size and stage. When a team member asks "why not X?", the answer is in this table.

Frontend

LAYER	TECHNOLOGY	VERSION	WHY THIS, NOT ALTERNATIVES
Framework	React	18.x	Concurrent mode, massive ecosystem, team expertise. Next.js rejected — SSR not needed for authenticated app; over-engineered for this use case.
Build Tool	Vite	5.x	10× faster HMR than Webpack. Native ESM. Module Federation plugin available.
MFE	@originjs/vite-plugin-federation	1.x	Webpack 5 MF spec, Vite-native. Each of 12 MFEs independently deployed to its own S3 + CloudFront. Single-SPA rejected — too opinionated.
Routing	React Router	7.x	File-based routing, lazy loading, data loaders. Works across MFE shell boundaries.
Styling	TailwindCSS	4.x	Zero unused CSS via JIT. Dark mode built-in — needed for warehouse/field low-light use.
Components	shadcn/ui + Radix UI	Latest	Accessible, copy-paste, Tailwind-native, no vendor lock-in. Ant Design rejected — opinionated, hard to override.
Server State	TanStack Query	5.x	Caching, background sync, optimistic updates. Cross-MFE cache sharing via singleton QueryClient.
Client State	Zustand	5.x	60 lines vs Redux 600. No providers. Cross-MFE shared state (tenant, user, feature flags) via singleton store.
Forms	React Hook Form + Zod	v7/v3	Uncontrolled = fast. Zod schemas shared with backend — single validation truth for quotation, customer, lead forms.

LAYER	TECHNOLOGY	VERSION	WHY THIS, NOT ALTERNATIVES
Tables	TanStack Table	8.x	Virtualised rows — handles 1M+ serial numbers in inventory view without lag.
Charts	Recharts	2.x	Composable, React-native. Best for CRM dashboards (pipeline funnel, revenue trend, stock movement).
Drag & Drop	dnd-kit	6.x	Pipeline Kanban board. Accessible. Touch + mouse for tablet warehouse use.
PDF Preview	react-pdf	7.x	In-browser quotation preview before send. No extra tab or external tool.
Barcode UI	@zxing/library	Latest	WebRTC camera for QR/barcode scanning in browser warehouse module. USB HID hardware scanner also supported.

Backend

WORKLOAD	FRAMEWORK	RUNTIME	WHY THIS SPECIFICALLY
Lambda functions (15 services: Leads, Customers, Enquiry, Quotation, Sales Order, Dispatch, GST, Inventory, Purchase, WhatsApp, Notifications, Audit, Reporting, Billing, Webhook)	Hono v4	Node.js 20 LTS	14KB bundle. 150ms cold start vs NestJS 3–8s. Web Standard Request/Response. Critical for Razorpay/WhatsApp webhooks with 3s Meta timeout.
Always-on containers (Auth, Invoicing)	Fastify v5	Node.js 20 LTS	30–40% faster than Express. Persistent DB connections. No cold start. GSTN IRP sessions need warm connections.
AI service (Lead scoring, forecasting, churn, OCR)	FastAPI + Pydantic v2	Python 3.12	Non-negotiable. LangChain, scikit-learn, sentence-transformers all Python-native. Rejected Node ML — ecosystem too immature.
Mobile GraphQL gateway	Mercurius (Fastify plugin)	Node.js 20 LTS	GraphQL lets mobile clients request only needed fields — critical for 2G rural sites. WebSocket subscriptions for real-time stock.

Data Layer

DATABASE	TECHNOLOGY	USE CASE	WHEN
Primary DB	PostgreSQL 16+	All transactional data. Multi-tenant via schemas + RLS. JSONB for custom fields. FTS for customer/lead search. pgvector for AI embeddings. UUID v4 PKs.	Day 1
Cache	Redis 7+	Sessions, rate limiting, GST OAuth token cache, OTP (10-min TTL for portal), product master cache-aside, real-time stock pub/sub.	Day 1
File Store	AWS S3	Quotation PDFs, invoices, vendor invoices, LR docs, delivery proofs, model artifacts. Presigned URLs only — no public S3 access.	Day 1

Database	Technology	Use Case	When
Search	AWS OpenSearch	Full-text customer, serial number, invoice search at scale.	500K+ records
Analytics	ClickHouse	Columnar for cross-module reports: lead-to-cash funnel, inventory aging, salesperson performance.	Phase 6
Vectors	pgvector (PG extension)	AI embeddings for lead scoring, customer similarity. Reuses PostgreSQL — no new infra.	Phase 6
Mobile Offline	SQLite via WatermelonDB	Local storage for React Native. Auto-sync with conflict resolution on reconnect.	Phase 5

Infrastructure

Component	Service	Why
IaC	AWS CDK v2 TypeScript — CloudArc base	TypeScript throughout. No state file. Day-0 AWS feature support. CloudArc extended, not replaced.
Lambda	AWS Lambda Node.js 20	Pay-per-invocation. Auto-scale 0→1000. Zero idle cost. Burst workloads: lead capture, quotation, stock queries.
Containers	AWS App Runner	Zero Kubernetes. Auto-scale. Deploy from ECR. Auth, Invoicing, AI, Mobile GQL need always-warm connections.
DB	AWS RDS PostgreSQL Multi-AZ	Managed. Automated backups. RDS Proxy for Lambda connection pooling.
Cache	AWS ElastiCache Redis	Managed. Automatic failover. Cluster mode at scale.
Queue	AWS SQS + SNS	Infinite scale. DLQs. FIFO for invoice generation — prevents duplicate GSTN submissions.
API Gateway	AWS API Gateway HTTP v2	JWT auth, rate limiting, WAF, plan-based throttling. \$1/M calls.
CDN	AWS CloudFront + S3	One distribution per MFE. Independent deployments. Presigned S3 URLs (15-min) for all documents.
Secrets	AWS Secrets Manager	GSTN API creds, WhatsApp tokens, Razorpay keys — all auto-rotated.
Monitoring	CloudWatch + Sentry + DataDog	CloudWatch: infra. Sentry: app errors. DataDog: APM for App Runner services.
ORM	Prisma v6	Schema-first. Type-safe. Multi-tenant schema support. Migration runner with rollback.
Push	AWS SNS + Firebase FCM	FCM for React Native iOS/Android. SNS as fan-out from backend to FCM.
Email	AWS SES	₹0.09/1000 emails. Transactional for quotations, invoices, OTPs. Bounce handling built-in.

CHAPTER 02

System Architecture Overview

SolarOS follows a serverless-first, API-first, event-driven architecture. All layers are independently deployable. The diagram shows the full request path from a field executive's mobile through to a GSTN API call and back.

END-TO-END ARCHITECTURE — ALL LAYERS



CHAPTER 03

Repository Structure (CloudArc Base)

One private GitHub repository. Turborepo builds only what changed per PR. CloudArc lives in `packages/infra` — extended, not replaced. Internal packages are workspace deps (`@solaros/*`).

```
solaros/ ← github.com/techpotato-software/solaros #— APPLICATIONS
└─ apps/ | └─ shells/ | └─ shell-suite/ → solaros.in
(full platform shell) | └─ shell-portal/ → portal.solaros.in (customer self-service) | └─ mfes/ ←
12 independently deployable micro-frontends | └─ mfe-dashboard/ →
cdn.solaros.in/dashboard/remoteEntry.js | └─ mfe-leads/ → cdn.solaros.in/leads/remoteEntry.js | └─
```

```

mfe-customers/ → cdn.solaros.in/customers/remoteEntry.js | └─ mfe-pipeline/ →
cdn.solaros.in/pipeline/remoteEntry.js | └─ mfe-quotations/ →
cdn.solaros.in/quotations/remoteEntry.js | └─ mfe-sales/ → cdn.solaros.in/sales/remoteEntry.js | └─
mfe-dispatch/ → cdn.solaros.in/dispatch/remoteEntry.js | └─ mfe-inventory/ →
cdn.solaros.in/inventory/remoteEntry.js | └─ mfe-purchase/ → cdn.solaros.in/purchase/remoteEntry.js
| └─ mfe-reports/ → cdn.solaros.in/reports/remoteEntry.js | └─ mfe-portal/ →
cdn.solaros.in/portal/remoteEntry.js (customer) | └─ mfe-settings/ →
cdn.solaros.in/settings/remoteEntry.js #— MICROSERVICES ————— | └─
services/ | └─ auth-service/ ← App Runner (Fastify) – always-on | | └─ src/commands/ ← HTTP
handlers (CSR: Command layer) | | └─ src/services/ ← Business logic (pure TS – no HTTP/DB) | | └─
src/repositories/ ← Prisma + Redis (only DB access here) | | └─ src/plugins/ ← Fastify plugins: jwt,
cors, prisma | | └─ Dockerfile ← App Runner container build | | └─ apprunner.yaml | └─ leads-
service/ ← Lambda (Hono) – same CSR structure | | └─ src/commands/ | | └─ src/services/ | | └─
src/repositories/ | | └─ src/events/ ← SQS event publishers | | └─ src/middleware/ ← tenant.ts,
validate.ts, ratelimit.ts | | └─ src/index.ts ← Lambda handler entry point | └─ customers-service/
← Lambda (Hono) – PRD Module 1 | └─ enquiry-service/ ← Lambda (Hono) – PRD Module 3 | └─ quotation-
service/ ← Lambda (Hono) – PRD Module 4 | └─ sales-order-service/ ← Lambda (Hono) – PRD Module 5 |
└─ invoicing-service/ ← App Runner (Fastify) – GSTN session mgmt | └─ dispatch-service/ ← Lambda
(Hono) – PRD Module 6 | └─ gst-service/ ← Lambda (Hono) – GSTN API wrapper | └─ inventory-service/
← Lambda (Hono) – serial + pallet tracking | └─ purchase-service/ ← Lambda (Hono) – PRD Module 8 |
└─ whatsapp-service/ ← Lambda (Hono) – Meta Cloud API | └─ notification-service/ ← Lambda SQS
trigger | └─ reporting-service/ ← Lambda EventBridge + on-demand | └─ audit-service/ ← Lambda SQS
trigger – async append-only | └─ billing-service/ ← Lambda Razorpay webhooks (PCCU=1) | └─ pdf-
worker/ ← Lambda container (Puppeteer, 10GB) | └─ mobile-api/ ← App Runner (Fastify + Mercurius
GraphQL) | └─ ai-service/ ← App Runner (FastAPI / Python 3.12) #— SHARED PACKAGES
————— | └─ packages/ | └─ types/ ← @solaros/types – all TypeScript
interfaces | | └─ src/entities/ Lead, Customer, Quotation, SalesOrder, Serial... | └─ ui/ ←
@solaros/ui – design system (MFE singleton) | └─ utils/ ← @solaros/utils | | └─ gst.ts ←
CGST/SGST/IGST, HSN lookup, IRN helpers | | └─ solar.ts ← Watt-kW calc, pallet/serial helpers | |
└─ currency.ts ← INR formatting, rate/watt calculations | | └─ dates.ts ← Indian date formats,
fiscal year helpers | | └─ validators.ts ← GSTIN, PAN, phone, IFSC, serial validators | └─ db/ ←
@solaros/db – Prisma client + schema | | └─ prisma/schema.prisma ← single source of truth for all
tables | | └─ prisma/migrations/ ← versioned migration files | └─ auth-client/ ← JWT RS256 verify +
tenant middleware | └─ queue/ ← SQS publisher + consumer helpers | └─ config/ ← Zod-parsed env vars
– fail on startup if missing | └─ infra/ ← @solaros/infra – CloudArc CDK (your boilerplate) | └─
src/stacks/ | └─ NetworkStack.ts | └─ DataStack.ts ← RDS + RDS Proxy + ElastiCache | └─
StorageStack.ts ← S3 + 12 CloudFront distributions | └─ ServicesStack.ts ← All Lambda + App Runner |
└─ ApiGatewayStack.ts | └─ QueueStack.ts ← SQS + SNS + EventBridge + DLQs | └─ MonitoringStack.ts
#— CONFIG ————— | └─ docker-compose.yml ← PostgreSQL + Redis +
LocalStack + MailHog | └─ turbo.json ← build/test/deploy pipelines | └─ pnpm-workspace.yaml ← workspace
discovery | └─ .github/workflows/ | └─ ci.yml ← lint + test + typecheck (Turbo affected) | └─
deploy-mfe.yml ← changed MFEs → S3 + CF invalidate | └─ deploy-service.yml ← changed services →
Lambda / App Runner | └─ deploy-infra.yml ← CDK diff + deploy (manual approval prod) | └─ mobile-
build.yml ← EAS Build + Submit + CodePush OTA | └─ README.md

```

CHAPTER 04

Microservices Design

All services follow the **CSR pattern** from CloudArc: **Command** (HTTP routing + Zod validation only), **Service** (pure business logic, no I/O — 100% unit-testable), **Repository** (only place Prisma and Redis are imported). Business rules are isolated and independently testable with no infra mocking.

SERVICE	COMPUTE	REASON FOR COMPUTE CHOICE	COLD START	EST. ₹/MONTH
auth-service	App Runner	JWT must respond in <50ms. Persistent JWKS cache. No cold start tolerance. 1–10 instances, CPU auto-scale.	None	₹2,000–6,000
invoicing-service	App Runner	E-Invoice IRN calls to GSTN need warm session window. PDF generation is CPU-heavy. Persistent DB connections. 1–20 instances.	None	₹3,000–10,000
ai-service	App Runner	Python ML models need warm memory. LangChain agents need persistent context. 2vCPU 4GB RAM. 1–5 instances.	None	₹4,000–15,000
mobile-api	App Runner	GraphQL subscriptions (WebSocket) require persistent process. Mobile clients hold connections for real-time stock updates.	None	₹2,000–8,000
leads-service	Lambda	Bursty CRUD during business hours. Hono cold start 150ms. Scales to 1000 concurrent free of charge.	150ms	₹200–800
customers-service	Lambda	CRUD + GSTIN duplicate check. Low idle between hours. RDS Proxy handles burst connections.	150ms	₹200–600
enquiry-service	Lambda	Enquiry writes follow sales hours. Follow-up log appends are lightweight writes.	150ms	₹100–400
quotation-service	Lambda	Floor-price check is pure business logic. kW calculation is compute, not I/O. Version optimistic lock on save.	150ms	₹200–600
sales-order-service	Lambda	SO creation bursty. FOR UPDATE stock reservation via RDS Proxy — Lambda-safe atomic row lock.	150ms	₹200–600
dispatch-service	Lambda	Triggered by payment events, not continuous. Delegates E-Invoice generation to gst-service via SQS FIFO.	150ms	₹100–400
gst-service	Lambda	Stateless GSTN API wrapper. Token cached in Redis. Retries handled via DLQ — no Lambda needs to stay alive.	150ms	₹100–300
inventory-service	Lambda	Stock checks bursty (order confirmations, dispatch). FOR UPDATE atomic — safe via RDS Proxy.	150ms	₹200–600
purchase-service	Lambda	PO creation low-frequency. GRN linking async via SQS. No long-held DB connections needed.	150ms	₹100–300
whatsapp-service	Lambda	Meta webhook 3s timeout — Hono 150ms cold start is safe. Inbound messages queued async to SQS.	150ms	₹100–400
notification-service	Lambda SQS	Async. No latency requirement. Routes to: WhatsApp SES email SMS FCM push based on channel field.	Irrelevant	₹100–400

SERVICE	COMPUTE	REASON FOR COMPUTE CHOICE	COLD START	EST. ₹/MONTH
billing-service	Lambda PCCU	Razorpay webhook 3s timeout. 1 provisioned concurrency = always warm, 0ms. Never miss a payment event.	0ms warm	₹500–1,200
pdf-worker	Lambda container	Puppeteer needs 1–3GB RAM. Lambda container supports up to 10GB. Zero idle cost between invoice bursts.	800ms	₹300–1,000
audit-service	Lambda SQS	Async. Append-only. RLS at DB level blocks DELETE on audit_logs — immutable even if service is compromised.	Irrelevant	₹100–300
reporting-service	Lambda	User-triggered. EventBridge for scheduled weekly reports. ClickHouse for heavy cross-module aggregations.	150ms	₹200–800

CHAPTER 05

Micro-Frontend Architecture

The shell app boots, authenticates the user, and lazy-loads each MFE's `remoteEntry.js` from its own CloudFront distribution on first route visit. Zero bundle sharing except declared singletons. A quotation module change requires zero downtime on the warehouse module — they deploy fully independently.

MFE Map — PRD Module Coverage

MFE	ROUTE	PRD MODULES COVERED	ROLES WITH ACCESS
mfe-dashboard	/dashboard	Quick stats, pipeline summary, stock alert count, AI lead scores	All internal roles
mfe-leads	/leads	Module 2 — Lead Management (capture, lifecycle, WhatsApp chat, convert)	Lead Manager, Sales Person, Sales Manager
mfe-customers	/customers	Module 1 — Customer Management (multi-contact, GSTIN, portal invite, reassign)	Sales Person, Sales Manager, Admin
mfe-pipeline	/pipeline	Module 3 — Enquiry Management (Kanban board, follow-up log, product interest)	Sales Person, Sales Manager
mfe-quotations	/quotations	Module 4 — Quotation (kW auto-calc, floor-price approval, PDF preview, versioning)	Sales Person, Sales Manager, Accounts
mfe-sales	/orders	Module 5 — Sales Order + Payment Milestones + Accounts Approval flow	Sales Person, Sales Manager, Accounts
mfe-dispatch	/dispatch	Module 6 — Dispatch + E-Invoice + E-way Bill + Serial Assignment + Document sharing	Dispatch Executive, Accounts

MFE	ROUTE	PRD MODULES COVERED	ROLES WITH ACCESS
mfe-inventory	/inventory	Module 7 — Warehouse Inward/Outward, Pallet/Serial tracking, Stock position	Warehouse Executive, Dispatch, Admin
mfe-purchase	/purchase	Module 8 — Purchase Orders, Vendor Management, GRN Link, FOR flow	Purchase Executive, Admin
mfe-reports	/reports	Module 10 — 16 pre-built cross-module reports, ClickHouse dashboards, export	Sales Manager, Admin
mfe-portal	/portal/*	Module 11 — Customer Portal (OTP login, orders, invoices, serials, support)	Customer (portal user only)
mfe-settings	/settings	Modules 9, 12, 13 — Products, Roles, Users, System Config, number series	Super Admin

Shared Singleton Strategy

Critical: Singletons loaded once in shell, marked `singleton: true` in Vite Federation config. Any MFE declaring the same package reuses the shell copy. Breaking this causes duplicate React contexts, dual query caches, and hook errors. Enforced via automated bundle analysis check in CI on every PR — build fails if a singleton is duplicated.

PACKAGE	WHY SINGLETON
react + react-dom	React hook rules require exactly one instance — any duplicate causes "Invalid hook call" errors across MFE boundaries
react-router-dom	Single browser history — nested routes work across MFEs from one router context
@tanstack/react-query	Shared QueryClient — different MFEs share cache, invalidation, and background sync
zustand	Shared global stores: tenant context, user profile, feature flags, notification state
@solaros/ui	Design system — one copy prevents duplicated CSS variables and component tree conflicts

CHAPTER 06

Database Schema — All Tables

Isolation: Schema-per-tenant in PostgreSQL. `public` schema holds only `tenants` and `feature_flags` . Every other table lives in `tenant_{uuid}` . Row Level Security is active on all tables, enforced independently of application code. Audit log has RLS blocking DELETE and UPDATE — append-only at the database level.

Conventions: All PKs are UUID (`gen_random_uuid()`). Timestamps are `TIMESTAMPTZ`. Soft deletes: `deleted_at TIMESTAMPTZ NULL` — never hard-delete business records. Optimistic locking: `version INT` incremented on every UPDATE. All money: `NUMERIC(12,2)` — never `FLOAT`.

tenants public schema (shared – all tenants)			
id	UUID	PK	gen_random_uuid()
name	TEXT		Company legal name
gstin	CHAR(15)	UNIQUE	Validated against GSTN format regex at insert
plan	TEXT		starter professional enterprise premium
schema_version	INT		DEFAULT 1 – rolling migration tracking per tenant
modules_enabled	JSONB		Array of active module keys: ["leads","quotations","dispatch"]
config	JSONB		Floor price %, quotation validity days, number series prefixes, GST config, WhatsApp template IDs
subscription_status	TEXT		active grace_period suspended cancelled
created_at	TIMESTAMPTZ		DEFAULT NOW()

users tenant_{uuid} – PRD Module 13			
id	UUID	PK	
email	TEXT	UNIQUE	Login identifier. Unique per tenant schema.
mobile	TEXT	IDX	AES-256-GCM encrypted. 10-digit India mobile.
role	TEXT	IDX	super_admin sales_manager sales_person accounts dispatch warehouse purchase customer
department	TEXT		Sales Accounts Warehouse Dispatch Purchase
reporting_manager_id	UUID	FK	→ users.id self-referential
verticals_assigned	TEXT[]		EPC Developer Investor Reseller Dealer
is_active	BOOLEAN		Inactive = cannot login; data preserved for audit
device_tokens	TEXT[]		FCM push tokens per device (iOS/Android)
last_login_at / created_at	TIMESTAMPTZ		

customers tenant_{uuid} – PRD Module 1			
id	UUID	PK	
company_type	TEXT	IDX	Individual Business
organisation_name	TEXT		Required if company_type = Business
primary_name	TEXT		Primary contact name. NOT NULL.
primary_email	TEXT	UNIQUE IDX	Portal login identifier
primary_phone	TEXT	IDX	Duplicate-checked on save. AES-256-GCM encrypted.

additional_contacts	JSONB		Array of {salutation, first_name, last_name, phone, email, designation}. Up to 10.
business_address	JSONB		{country, state, district, city, region, pincode}
shipping_address	JSONB		NULL = same as business_address
gst_in	TEXT	IDX	15-char, regex-validated. Required if Business. Encrypted. Validated against GSTN API on save.
pan_number	TEXT		10-char PAN. Encrypted at rest.
company_category	TEXT	IDX	A B C D E — criteria configured in tenant.config
industry_type	TEXT	IDX	EPC Developer Investor Reseller Dealer Other
lead_channel	TEXT		LinkedIn Facebook Instagram Direct Google Indiamart Other
assigned_sales_person_id	UUID	FK	→ users.id. Re-assign only by manager role. Notification sent on change.
portal_invited	BOOLEAN		DEFAULT false. Set true when portal invite WhatsApp sent.
status	TEXT	IDX	Active Inactive Blacklisted
internal_notes	TEXT		Rich text. Not visible in customer portal.
custom_data	JSONB		kw_requirement, discom_zone, project_location — vertical-specific fields
version	INT		DEFAULT 1. Optimistic locking — increment on every UPDATE.
created_at / deleted_at	TIMESTAMP TZ		Soft delete — blocked if open enquiries, quotations, or orders exist

Leads tenant_{uuid} — PRD Module 2			
id	UUID	PK	Display: LEM-YYYY-NNNNNN (generated in service layer)
source	TEXT	IDX	Call WhatsApp LinkedIn Facebook Instagram Google Indiamart Other
industry_type	TEXT	IDX	EPC Developer Investor Reseller Dealer Other
contact_name / phone / email	TEXT		phone: duplicate-checked across leads + customers on insert
status	TEXT	IDX	New In-Progress Follow-Up Qualified Converted Closed-Lost
ai_score	SMALLINT	IDX DESC	0–100. Set by ai-service on lead.created event. Updated on each interaction.
assigned_to	UUID	FK IDX	→ users.id. WhatsApp notification sent on assignment.
purchase_timeline	TEXT		Immediate 1-3 months 3-6 months 6+ months
estimated_quantity	INT		Number of panels
follow_up_log	JSONB		Array of {logged_by, logged_at, type (Call WhatsApp Email Meeting Visit), remarks, next_follow_up}
lost_reason	TEXT		Required when status = Closed-Lost
converted_customer_id	UUID	FK	→ customers.id. Set on conversion. Lead record preserved and linked.
custom_data	JSONB		product_interest, location, discom_zone, roof_type

version / created_at /
deleted_at

STANDARD

Soft delete. Never hard-delete leads.

enquiries tenant_{uuid} – PRD Module 3

id	UUID	PK	Display: ENQ-YYYY-NNNNNN
customer_id	UUID	FK IDX	→ customers.id. Always linked to verified customer.
assigned_sales_person_id	UUID	FK	→ users.id. Auto from customer, overridable by manager.
line_items	JSONB		Array of {product_id, quantity, customer_price_ask, remarks}
status	TEXT	IDX	Created In-Progress Follow-Up Closed-Won Closed-Lost
lost_reason	TEXT		Required when Closed-Lost
follow_up_log	JSONB		Array of {date, user_id, type, remarks, next_follow_up_date}
version / created_at / deleted_at	STANDARD		

quotations tenant_{uuid} – PRD Module 4

id	UUID	PK	Display: QUO-YYYY-NNNNNN (editable by sales person before Sent status)
display_id	TEXT	IDX	Editable for informal quotes. Audit log stores original + modified ID.
customer_id / enquiry_id / sales_person_id	UUID	FK	→ customers, enquiries, users
valid_until	DATE		DEFAULT: creation_date + config.quotation_validity_days (configurable per tenant)
line_items	JSONB		Array of {product_id, qty_panels, wattage_wp, qty_kw, rate_per_watt, rate_per_panel, discount_pct, gst_pct, net_line_value}. Computed on save.
subtotal / total_gst / grand_total	NUMERIC(12,2)		Stored computed values. Recalculated by service layer on every save.
total_kw / total_panels	NUMERIC(10,3) / INT		Formula: (Wp × qty) / 1000 per line. Aggregated on save.
approval_status	TEXT	IDX	Not-Required Pending Approved Rejected. Auto-set when any line rate_per_watt < floor price.
approval_manager_id / approval_remarks	UUID / TEXT		Who approved/rejected + comment shown to sales person
status	TEXT	IDX	Draft Pending-Approval Sent Accepted Rejected Expired Converted
terms_and_conditions	TEXT		Pre-filled from config template, editable per quotation
version / created_at / deleted_at	STANDARD		Each revision = new version row. Old versions preserved permanently.

sales_orders tenant_{uuid} – PRD Module 5			
id	UUID	PK	Display: SO-YYYY-NNNNNN
quotation_id / customer_id	UUID	FK IDX	Products + pricing pulled from approved quotation on creation
payment_milestones	JSONB		Array of {name, amount, due_date, mode, reference, status, approved_by, approved_at, proof_s3_key}
advance_approval_status	TEXT	IDX	Pending Approved Rejected — set by accounts team
balance_payment_status	TEXT		Pending Received Approved
product_allocation_status	TEXT	IDX	Pending-Stock Allocated Partially-Allocated. Updated when inventory reserves stock.
status	TEXT	IDX	Draft Active Dispatched Delivered Completed Cancelled
expected_delivery_date	DATE		Optional customer-facing ETA
version / created_at	STANDARD		

dispatch_records tenant_{uuid} – PRD Module 6			
id	UUID	PK	Display: DIS-YYYY-NNNNNN
sales_order_id	UUID	FK IDX	→ sales_orders.id
invoice_number	TEXT	UNIQUE	Sequential from number series in tenant.config
serial_numbers_dispatched	TEXT[]		Array of individual panel serials. Validated against allocated stock on save.
vehicle_number / driver_name / driver_contact	TEXT		Required for EX-Works. NULL for FOR delivery.
lr_document_s3_key	TEXT		S3 key for uploaded LR PDF
e_invoice_irn	TEXT	IDX	Invoice Reference Number from GSTN IRP. Set after gst-service succeeds.
e_invoice_qr_code	TEXT		Base64 QR from GSTN. Printed on invoice PDF.
eway_bill_number / eway_bill_expiry	TEXT / TIMESTAMPTZ		Generated after IRN. EventBridge alert 24h before expiry.
delivery_proof_s3_key	TEXT		POD document. Set when status = Delivered.
status	TEXT	IDX	Prepared Dispatched In-Transit Delivered
is_for_delivery	BOOLEAN		TRUE = vendor-arranged transport. Vehicle fields NULL. FOR special flow applies.

inventory_items tenant_{uuid} – PRD Module 9 (Product Master)			
id / sku	UUID / TEXT	PK / UNIQUE	SKU auto-generated from category + brand + wattage
category / sub_category	TEXT	IDX	Solar Panels Inverters Mounting Structures Cables Accessories
brand	TEXT		Waaree Adani Havells etc.

watt_peak	NUMERIC(6,2)	e.g. 540.00 Wp. Required for solar panels. Used in kW calc: (Wp × qty) / 1000.
hsn_code	CHAR(8)	8541 for solar panels. 8504 for inverters. Required for E-Invoice submission.
gst_rate_pct	NUMERIC(4,2)	5 12 18 28. Auto-applied in quotation and invoice.
floor_price_per_watt	NUMERIC(8,4)	If quotation rate_per_watt < this → approval required. Set by sales manager.
is_serial_tracked	BOOLEAN	TRUE for solar panels. FALSE for cables, accessories.
is_pallet_tracked	BOOLEAN	TRUE if serial tracked and uses pallets (mother serial enabled).
panels_per_pallet_default	INT	DEFAULT 30. Overridable at inward entry.
quantity / reserved	NUMERIC(10,2)	Updated with FOR UPDATE row lock via RDS Proxy on every stock mutation.
available	NUMERIC(10,2)	GENERATED ALWAYS AS (quantity - reserved). Cannot be set directly.
reorder_level	NUMERIC(10,2)	WhatsApp alert to purchase team when available ≤ reorder_level
almm_status	TEXT	listed de-listed not-applicable. Government tender compliance.

serial_numbers tenant_{uuid} – individual panel traceability			
id	UUID	PK	
serial_number	TEXT	UNIQUE IDX	Individual panel serial barcode. Unique per tenant.
pallet_id	UUID	FK IDX	→ pallets.id. The mother pallet this panel belongs to.
inventory_item_id / warehouse_id	UUID	FK IDX	→ inventory_items.id, warehouses.id — current product and location
status	TEXT	IDX	In-Stock Reserved Dispatched Returned Damaged Lost
physical_condition	TEXT		Good Minor-Damage Major-Damage. Set at inward scanning.
reserved_for_so_id	UUID	FK	→ sales_orders.id. Set when advance payment approved by accounts.
dispatched_in_dispatch_id	UUID	FK	→ dispatch_records.id. Enables full purchase→customer traceability chain.
purchase_order_id	UUID	FK	→ purchase_orders.id. Panel → PO → vendor traceability.
damage_notes	TEXT		Required if condition ≠ Good. Triggers vendor claim flow.
received_at / dispatched_at	TIMESTAMP TZ		Full lifecycle timestamps per panel.

pallets tenant_{uuid} – mother serial / pallet QR			
id	UUID	PK	
mother_serial	TEXT	UNIQUE IDX	Pallet QR barcode. Scanning this auto-reads all child serials if vendor supports.

inward_id	UUID	FK	→ inward_records.id. Which GRN this pallet arrived on.
panels_count	INT		Actual panels received (may differ from default 30).
physical_condition	TEXT		Good Minor-Damage Major-Damage. Per-pallet condition at inward.

inward_records tenant_{uuid} – PRD Module 7 Inward (GRN)			
id	UUID	PK	Display: INW-YYYY-NNNNNN
inward_type	TEXT	IDX	Purchase-EXW Purchase-FOR Stock-Transfer Vendor-Replacement Customer-Return-Full Customer-Return-Partial
warehouse_id / purchase_order_id	UUID	FK IDX	Where it's going + which PO it fulfils
vendor_invoice_number / vendor_invoice_date	TEXT / DATE		Required. Cross-referenced with PO line items.
vendor_invoice_s3_key	TEXT		S3 key for uploaded vendor invoice PDF
qty_expected / qty_received	INT		Discrepancy triggers flag to purchase team if they differ
vehicle_number / driver_name	TEXT		Captured at warehouse gate for EX-Works deliveries
documents_collected	JSONB		{vendor_invoice: bool, lr_grn: bool, packing_list: bool}
received_by / received_at	UUID / TIMESTAMPTZ		Auto-set from logged-in warehouse user session
grn_remarks	TEXT		Warehouse notes on receipt condition

purchase_orders tenant_{uuid} – PRD Module 8			
id	UUID	PK	Display: PO-YYYY-NNNNNN
vendor_id	UUID	FK IDX	→ vendors.id (vendor master)
delivery_type	TEXT		EX-Works FOR. Determines transport responsibility and dispatch flow.
delivery_destination	JSONB		{type: warehouse customer_address, warehouse_id?, address?}. FOR can deliver directly to end customer.
line_items / po_total	JSONB / NUMERIC(14,2)		Array of {product_id, qty, rate_per_unit, gst_pct, line_total}. Total calculated on save.
status	TEXT	IDX	Draft Sent Partially-Received Fully-Received Closed
linked_inward_ids	UUID[]		Auto-populated as warehouse records inward entries against this PO

audit_logs tenant_{uuid} – append-only (RLS blocks DELETE + UPDATE)			
id	UUID	PK	
user_id / tenant_id	UUID	FK	Who performed the action

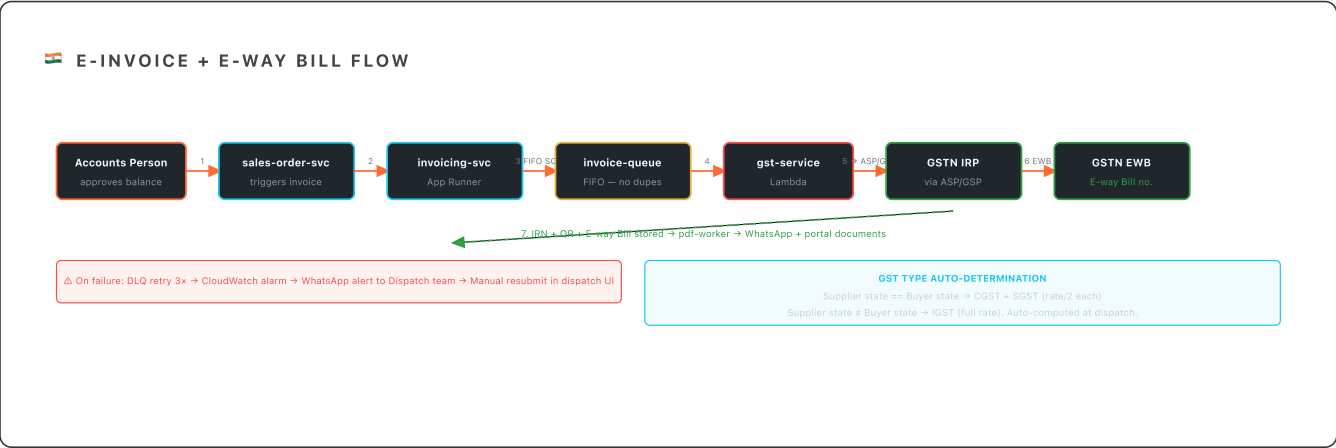
action	TEXT	IDX	CREATE UPDATE DELETE APPROVE REJECT LOGIN EXPORT PORTAL_VIEW
entity_type / entity_id	TEXT / UUID	IDX	What changed — customers, leads, quotations, serial_numbers, etc.
old_value / new_value	JSONB		Field-level delta. Sensitive fields shown as [ENCRYPTED] in audit UI.
ip_address	INET		Client IP at time of action
created_at	TIMESTAMPTZ	IDX	DEFAULT NOW(). RLS policy FOR UPDATE USING (false) — cannot be modified.

CHAPTER 07

GST Integration Architecture

GST compliance is handled by the dedicated `gst-service` Lambda — the system's sole interface to GSTN. This service wraps the ASP/GSP layer, manages OAuth2 tokens (cached in Redis), handles retries via DLQ, and stores all API responses. The invoicing-service never calls GSTN directly — always through `gst-service` via SQS FIFO queue to prevent duplicate E-Invoice submissions.

ASP/GSP Requirement: SolarOS connects through a certified GSP (ClearTax, Masters India, or Tally Solutions) — not directly to GSTN IRP. The GSP provides OAuth2 tokens and REST wrapper. GSP credentials stored in Secrets Manager with auto-rotation.
Start GSP onboarding in Phase 2 — GSTN approval takes 2–3 weeks.



FUNCTION	DETAILS	GSTN ENDPOINT
GSP Token Refresh	OAuth2 token cached in Redis. Auto-refreshed 5 min before expiry. Failure → PagerDuty alert.	ASP/GSP OAuth
E-Invoice (IRN)	Posts invoice JSON per GSTN schema v1.1. Stores IRN + signed QR + signed JSON. FIFO queue prevents duplicate submission on retry.	POST /generateIRN
E-way Bill	Generated after IRN. Requires vehicle number (EX-Works) or transporter GSTIN (FOR). Validity stored + EventBridge alert 24h before expiry.	POST /generateEwb
IRN Cancel	Within 24-hour window only. Reason stored. Dispatch record updated. New IRN required if resubmitted.	POST /cancelIRN

FUNCTION	DETAILS	GSTN ENDPOINT
E-way Bill Extension	Dispatch team extends before expiry via UI. New expiry stored in dispatch_records.	PUT /updatePartB
GSTIN Validation	Validates customer GSTIN at save. Cached in Redis 24h. Returns registered trade name to pre-fill organisation_name.	GET /search/gstin

CHAPTER 08

WhatsApp Integration Architecture

SolarOS uses the **Meta Cloud API** for all messaging. The [whatsapp-service](#) Lambda is the system's sole Meta interface. All outbound sends are async — services publish to [notification-queue](#) SQS, notification-service routes to whatsapp-service. Inbound messages route via whatsapp-service → SQS fan-out → leads-service (new lead) or ai-service (intent classification).

TEMPLATE	TRIGGER EVENT	KEY VARIABLES	TYPE
lead_assigned	Lead assigned to sales person	salesperson_name, lead_name, follow_up_date	Internal
quotation_sent	Quotation PDF shared with customer	customer_name, quotation_id, valid_until, total_amount, pdf_link	Transactional
advance_payment_reminder	EventBridge daily cron — due date minus 2 days	customer_name, so_id, amount, due_date, payment_link	Reminder
payment_confirmed	Accounts approves payment milestone	customer_name, so_id, amount, payment_date	Transactional
dispatch_initiated	Dispatch record created + documents ready	customer_name, invoice_no, invoice_link, eway_bill_no	Transactional
in_transit_update	Dispatch status → In Transit	customer_name, vehicle_no, driver_contact, eway_bill_no, expected_delivery	Transactional
delivery_confirmed	Dispatch status → Delivered	customer_name, so_id, delivered_date, portal_link	Transactional
portal_otp	Customer portal login request	customer_name, otp (6-digit), validity_minutes (10)	Auth
low_stock_alert	available ≤ reorder_level	product_name, warehouse, available_qty, reorder_level	Alert
quotation_approval_needed	Rate below floor price — quotation locked	manager_name, sales_person, customer, rate, floor_rate, deviation_pct	Alert
eway_bill_expiring	EventBridge 24h before eway_bill_expiry	dispatch_exec_name, invoice_no, eway_bill_no, expiry_time	Alert

Meta 24-Hour Window: WhatsApp only allows free-form messages to customers who messaged first within 24 hours. Outside this window, only approved template messages can be sent. SolarOS tracks last-inbound timestamp per customer phone in `whatsapp_messages` . whatsapp-service auto-selects template vs free-form based on this timestamp check before every send.

CHAPTER 09

Warehouse & Serial Tracking Architecture

Every solar panel in a SolarOS warehouse is tracked at two levels: **mother serial** (pallet QR) and **child serial** (individual panel barcode). The system maintains an unbroken traceability chain from purchase order → GRN → stock reservation → dispatch → customer delivery — fully queryable via the Serial Number Traceability Report.

**Mother Serial (Pallet QR)**

Each pallet has one QR sticker from the manufacturer (e.g., Waaree). Scanning the pallet QR in the warehouse app auto-populates all child serials if the vendor's QR encodes structured data. Stored in `pallets.mother_serial` . Default 30 panels per pallet — configurable per product.

**Child Serials (Panel Barcodes)**

Each panel has its own barcode. If pallet QR doesn't auto-populate child serials, the warehouse executive scans each panel individually or pastes a CSV from the vendor packing list. Stored in `serial_numbers` table linked to `pallets.id` and `inventory_items.id` .

**Real-Time Stock Formula**

Available = Quantity – Reserved
Closing = Opening + Total Inward – Total Outward
Maintained per product SKU per warehouse. Updated via `FOR UPDATE` row-level locks on `inventory_items` using RDS Proxy — Lambda-safe atomic mutations.

INWARD TYPE	SOURCE DOCUMENT	SERIAL REQUIREMENT	SCANNING MODE
Purchase — EX Works	Linked PO + Vendor Invoice	Mother serial + all child panel serials	Pallet QR first, then individual panel barcodes
Purchase — FOR (Vendor delivers to Oriana WH)	Vendor Invoice uploaded by purchase team	Same as EX Works	Same scanning flow + vehicle details at gate
Stock Transfer (inter-warehouse)	Internal Transfer Note	Serial list carried over from source	Confirm receipt by scanning subset to verify
Vendor Replacement	Replacement Advice from vendor	New serials in; damaged serials out	Scan new panels in + confirm old removed
Customer Return — Full	Credit Note reference	All returned serials re-stocked	Scan each returned panel. Condition noted.
Customer Return — Partial	Credit Note reference	Returned serials only. Rest still at customer.	Scan returned panels. Partial credit note created.

FOR Delivery (Vendor → Customer Direct): When Waaree delivers 400 panels directly to Oriana's customer (bypassing Oriana warehouse): (1) Purchase team uploads Waaree's delivery invoice into the PO. (2) Links invoice to the customer SO. (3) Dispatch

team shares docs via portal + WhatsApp. (4) No inward_record created — stock never enters Oriana warehouse. (5) Panel serials recorded directly in `serial_numbers` with status Dispatched, linked to dispatch_record.

CHAPTER 10

Mobile App Architecture

SolarOS ships three React Native apps for the three most common field roles. All three share one codebase — role-based feature flags enable/disable screens at runtime. Offline-first architecture ensures field executives in poor-connectivity sites (rural solar projects) can work fully and sync when reconnected.

CONCERN	DECISION	RATIONALE
Framework	React Native (Expo managed)	Shared TypeScript types from <code>@solaros/types</code> . Single codebase for iOS + Android. Expo EAS for cloud builds + OTA updates without app store review.
API Protocol	GraphQL via Mercurius (Fastify App Runner)	Mobile requests only needed fields — reduces payload 70–80% vs REST. Critical for 2G rural sites. Subscriptions for real-time stock.
Auth	Same JWT RS256 as web. Biometric unlock (keychain/keystore) releases cached token.	No separate auth system. Same Lambda Authorizer validates. Face ID / Fingerprint for field speed.
Offline Storage	WatermelonDB (SQLite-backed)	Leads, customers, enquiry forms, GRN drafts all available offline. Auto-sync on reconnect.
Sync Conflicts	Last-write-wins for most fields. Server wins for approval statuses and stock quantities.	Sales notes can merge; approval state cannot — must come from authoritative server.
Push Notifications	Firebase Cloud Messaging (FCM) for iOS via APNs bridge and Android	Single SDK. Token stored in <code>users.device_tokens[]</code> . Managed by notification-service.
Barcode Scanning	react-native-vision-camera + MLKit Barcode Scanner	Fastest on both platforms. Works offline. USB HID hardware scanner also supported for heavy warehouse use.
Photo Capture	react-native-camera → compress → presigned S3 upload	Damage photos at inward, POD at delivery. WebP compressed to max 1MB before upload.
State Management	Redux Toolkit + Redux Persist	Redux Persist for offline state hydration on app open. Zustand singletons (web) cannot cross native bridge.
OTA Updates	Expo EAS Updates	Push JS bundle updates without App Store review cycle. Critical for GST rule changes mid-year.
Analytics	Firebase Analytics + Crashlytics	Session tracking, crash reports, screen funnel per role.
Maps	react-native-maps (Google Maps)	Delivery route planning for dispatch executive. Customer location pinning for field sales.

APP ROLE	KEY FEATURES	CRITICAL OFFLINE CAPABILITY
Field Executive App	Lead capture with photo + GPS location, customer follow-up call logs, enquiry creation, quotation PDF view, WhatsApp quick-share of documents	Leads and customers readable offline. New lead creation queued and synced on reconnect.
Warehouse Executive App	Pallet QR + panel barcode scanning, inward GRN creation, damage photo capture + notes, real-time stock visibility, stock transfer initiation	GRN drafts saveable offline. Serials scanned locally. Full sync on WiFi reconnect.
Dispatch Executive App	Active SO list, serial number confirmation, dispatch status updates, POD photo capture + customer signature, GPS delivery location stamp	Delivery confirmation and POD upload queued offline. Auto-uploads when network available.

CHAPTER 11

Event-Driven Architecture

All cross-service communication is asynchronous via SQS and SNS. No service calls another service synchronously through internal APIs. A failure in notification-service does not affect lead creation. Every queue has a Dead Letter Queue (`maxReceiveCount: 3`). DLQ messages trigger CloudWatch alarms + WhatsApp alert to on-call engineers.

lead-events SNS → SQS fan-out Standard DLQ ✓ PRODUCER leads-service on every lead CREATE / UPDATE / CONVERT CONSUMERS ai-service (score 0–100), notification-svc (WhatsApp to salesperson on assign), audit-svc (append log) KEY EVENTS lead.created, lead.updated, lead.converted, lead.score_updated, lead.follow_up_due	quotation-events SNS fan-out Standard DLQ ✓ PRODUCER quotation-service on create / approval trigger / status change CONSUMERS notification-svc (WhatsApp to manager for approval; to sales person on approve/reject), audit-svc KEY EVENTS quotation.approval_requested, quotation.approved, quotation.rejected, quotation.sent, quotation.expired
payment-events SQS FIFO DLQ ✓ — ORDER CRITICAL PRODUCER billing-service on Razorpay webhook receipt (HMAC verified first) CONSUMERS sales-order-svc (activate SO, update milestone), inventory-svc (reserve stock with FOR UPDATE lock) KEY EVENTS payment.captured, payment.approved, payment.rejected, balance.approved	invoice-queue SQS FIFO DLQ ✓ — NO DUPLICATE E-INVOICES PRODUCER invoicing-service after balance payment approval from accounts CONSUMERS gst-service (E-Invoice IRN → E-way Bill), pdf-worker (invoice PDF generation) KEY EVENTS invoice.generate_irn, invoice.irn_ready, invoice.eway_bill_ready, invoice.pdf_ready
dispatch-events SNS fan-out Standard DLQ ✓	inventory-events SNS fan-out Standard DLQ ✓

PRODUCER dispatch-service on status transitions CONSUMERS inventory-svc (serial status → Dispatched, outward stock), notification-svc (WhatsApp + email to customer), audit-svc KEY EVENTS dispatch.prepared, dispatch.dispatched, dispatch.in_transit, dispatch.delivered	PRODUCER inventory-service on every inward / outward / stock mutation CONSUMERS notification-svc (WhatsApp low-stock alert to purchase team), reporting-svc (stock position cache refresh) KEY EVENTS stock.inward_completed, stock.outward_completed, stock.low_alert, stock.discrepancy_flagged
notification-queue SQS Standard DLQ ✓ — universal output bus PRODUCER Any backend service that needs to send a notification to any user CONSUMERS notification-service → routes to: WhatsApp AWS SES email SMS FCM push based on channel field PAYLOAD {channel, recipient_id, template_name, variables, priority, idempotency_key}	audit-queue SQS Standard DLQ ✓ — every mutation audited PRODUCER Every service — every mutation publishes an audit event after successful write CONSUMERS audit-service → INSERT into audit_logs (append-only via RLS policy) NOTE DLQ critical — lost audit events cannot be reconstructed. PagerDuty alert on DLQ depth > 0.
pdf-queue SQS Standard DLQ ✓ PRODUCER invoicing-service (invoices), quotation-service (quotation PDFs) CONSUMERS pdf-worker Lambda (10GB Puppeteer container) → S3 upload → presigned URL stored in record EVENTS pdf.generate_invoice, pdf.generate_quotation, pdf.generate_packing_list, pdf.generate_challan	EventBridge Rules (4 cron rules) Scheduled triggers → SQS fan-out DAILY 9AM Payment reminder check → notification-queue for overdue milestones DAILY 8AM Follow-up reminder → sales person WhatsApp for today's scheduled follow-ups WEEKLY MON Salesperson performance report → email to sales manager E-WAY BILL Check expiry 24h before → WhatsApp alert to dispatch team for extension

DLQ Strategy: Every queue has a DLQ with `maxReceiveCount: 3` . After 3 failed processing attempts, message moves to DLQ and triggers a CloudWatch alarm + WhatsApp alert to on-call. For invoice-queue and payment-events (FIFO): DLQ messages are manually reviewed and replayed after root-cause fix — never auto-retried to avoid duplicate GSTN submissions or double-payments.

CHAPTER 12

Security Architecture — Zero-Trust

Five independent tenant isolation layers. Bypassing one does not bypass the next. Even a fully compromised application service cannot read another tenant's data — RLS operates at the database level, independent of all application code.

1

Network Perimeter — AWS WAF + TLS 1.3

All traffic enters via API Gateway HTTP v2 with WAF: OWASP Top 10, SQL injection, XSS, bot control. TLS 1.3 enforced at CloudFront and API Gateway. HSTS headers on all responses. Rate limiting by plan tier (Starter: 100 req/min, Enterprise: 5,000 req/min). No HTTP — any unencrypted request is rejected.

2 Authentication — Lambda Authorizer + JWT RS256

Every API call passes through the Lambda Authorizer before routing. Verifies JWT using RS256 public key from `auth-service/.well-known/jwks.json` (cached 15 minutes in authorizer memory). JWT payload: `{userId, tenantId, roles, entitlements, plan}`. Verified headers (`X-Tenant-Id`, `X-User-Id`, `X-Roles`) injected — services trust them implicitly. WAF blocks direct header injection attempts from external clients.

3 Application Layer — Tenant Context Middleware

Every Hono/Fastify service reads `X-Tenant-Id` from the verified request header. The `tenantMiddleware` from `@solaros/auth-client` executes `SET search_path TO tenant_{id}, public` on the PostgreSQL connection before any Prisma query. All subsequent queries in that request run scoped to the tenant schema. Service code never constructs schema names from user input — the tenant ID always comes from the verified JWT header.

4 Database — PostgreSQL RLS (Final Guard)

Row Level Security active on every table: `USING (tenant_id = current_setting('app.tenant_id')::uuid)`. RLS operates independently of application code — even if middleware is bypassed, RLS filters all rows silently. Audit log has: `FOR DELETE USING (false)` and `FOR UPDATE USING (false)` — append-only enforced at DB level. Lambda IAM roles: `SELECT/INSERT/UPDATE/DELETE` only — no DDL, no TRUNCATE, no DROP schema.

5 Data — PII Encryption + S3 Presigned URLs

Sensitive fields (phone, GSTIN, PAN, bank details) encrypted with AES-256-GCM using per-tenant KMS key before storage. RDS encrypted at rest (KMS). S3: SSE-S3. All document URLs presigned with 15-minute expiry — no public S3 URLs exist anywhere in the system. Customer portal sessions: 4-hour JWT + OTP refresh flow. All portal visits logged in `audit_logs`.

RBAC Matrix (PRD Module 12)

ROLE	LEADS	CUSTOMERS	QUOTATIONS	SALES ORDERS	DISPATCH	WAREHOUSE	PURCHASE	RE
Super Admin	CRUD All	CRUD All	CRUD All	CRUD All	CRUD All	CRUD All	CRUD All	Fu
Sales Manager	CRUD team	CRUD team, reassign	CRUD, Approve	R, Approve payment	R	R	R	Fu
Sales Person	CRUD (own)	CRUD (own)	CRU (own)	CR (own)	R (own SO)	R (stock only)	None	Ov ste
Accounts	R	R	R, Approve payment	R, Approve payment	Generate invoice	R	R	Fir
Dispatch Exec	None	R	R	R (active only)	CRUD	R (stock view)	R	Dis

ROLE	LEADS	CUSTOMERS	QUOTATIONS	SALES ORDERS	DISPATCH	WAREHOUSE	PURCHASE	RE
Warehouse Exec	None	None	None	R (booked stock)	R	CRUD	R (PO view)	Inv
Purchase Exec	None	None	None	None	R (FOR docs)	R	CRUD	Pu
Customer Portal	None	Own profile (R)	Own (R)	Own (R)	Own docs (R)	Own serials (R)	None	No

CHAPTER 13

CI/CD Pipeline

Turborepo detects which packages changed per PR and only builds/tests/deploys those. A change in `packages/types` triggers rebuilds of all 18 services that depend on it. A change in only `services/gst-service` rebuilds and deploys just that Lambda. Full repo parallel build: ~12 minutes. Single service: ~90 seconds.

WORKFLOW FILE	TRIGGER	STEPS	TARGET
ci.yml	Every PR + push to main	pnpm install → turbo lint → turbo typecheck → turbo test (affected) → turbo build (affected). Blocks merge on failure.	All changed packages and their dependents
deploy-mfe.yml	Merge to main (if any MFE changed)	turbo build (affected MFE only) → aws s3 sync dist/ → CloudFront create-invalidation → Playwright smoke tests on staging	Changed MFE S3 bucket + CloudFront distribution
deploy-service.yml	Merge to main (if any service changed)	turbo build (affected service) → Lambda: zip + update-function-code. App Runner: docker build → push ECR → update-service → wait for deployment	Changed Lambda or App Runner service
deploy-infra.yml	Manual trigger only (never on auto-merge)	cdk diff → post diff as PR comment → Manual Approval gate in GitHub Actions → cdk deploy (named target stack only)	Specified CDK stack (prod)
mobile-build.yml	Git tag push: mobile/v*	EAS Build (iOS + Android) → EAS Submit to App Store + Play Store → Expo EAS Update (OTA) for staged rollout	App Store + Google Play Store

Turborepo affected command: `pnpm turbo test --filter=...[HEAD^1]` — the `...[HEAD^1]` filter means "all packages changed vs previous commit plus all their dependents." Touching `@solaros/types` triggers tests for all 18 services. Touching only `services/whatsapp-service` triggers just that service's tests. Build outputs are content-addressed — unchanged packages are never rebuilt regardless of how many times CI runs.

CHAPTER 14

Module Deployment Map

PRD MODULE	BACKEND SERVICE(S)	COMPUTE	MFE	CORE DB TABLES
Module 1 — Customers	customers-service	Lambda	mfe-customers	customers
Module 2 — Leads	leads-service	Lambda	mfe-leads	leads
Module 3 — Enquiry	enquiry-service	Lambda	mfe-pipeline	enquiries
Module 4 — Quotations	quotation-service	Lambda	mfe-quotations	quotations
Module 5 — Sales Orders	sales-order-service	Lambda	mfe-sales	sales_orders
Module 6 — Dispatch + GST	dispatch-service + gst-service + invoicing-service	Lambda + App Runner	mfe-dispatch	dispatch_records, invoices
Module 7 — Warehouse	inventory-service	Lambda	mfe-inventory	warehouses, serial_numbers, pallets, inward_records, outward_records, stock_transactions
Module 8 — Purchase	purchase-service	Lambda	mfe-purchase	purchase_orders, vendors
Module 9 — Products	shared via inventory + quotation service	Lambda	mfe-settings (products section)	inventory_items
Module 10 — Reports	reporting-service	Lambda	mfe-reports	Read-only cross-module + ClickHouse for aggregations
Module 11 — Customer Portal	auth-service (OTP) + all read- only services	App Runner	mfe-portal	portal_sessions, portal_support_requests
Modules 12, 13 — Roles + Users + Config	auth-service	App Runner	mfe-settings	users, role_permissions, system_config
WhatsApp (cross-cutting)	whatsapp-service + notification-service	Lambda	Chat tab in mfe- leads, mfe- customers	whatsapp_messages, whatsapp_template_submissions
AI (cross-cutting)	ai-service	App Runner (Python 3.12)	Score badges in mfe-leads, mfe- pipeline, mfe- dashboard	ml_predictions, ai_model_versions

PRD MODULE	BACKEND SERVICE(S)	COMPUTE	MFE	CORE DB TABLES
Mobile Apps (iOS + Android)	mobile-api (GraphQL gateway)	App Runner	React Native apps (3 roles)	Reads all tables via GraphQL resolvers. WatermelonDB offline replica.

CHAPTER 15

CloudArc CDK Stack Structure

CloudArc (existing CDK boilerplate) is the foundation — extended, not replaced. Stack order must be followed: each stack exports outputs consumed by the next. All SolarOS L3 constructs extend CloudArc base constructs.

STACK	KEY RESOURCES	DEPENDS ON	EXPORTS
NetworkStack	VPC (3 AZs), 3 public + 3 private subnets, NAT Gateway, VPC Flow Logs, Security Groups for Lambda / App Runner / RDS / ElastiCache	None	vpclid, privateSubnetIds, securityGroupIds
DataStack	RDS PostgreSQL 16 Multi-AZ (db.t3.medium prod / db.t3.micro staging), RDS Proxy (connection pooling, Lambda-safe), ElastiCache Redis 7 (cache.t3.micro), DB credentials Secret	NetworkStack	rdsProxyEndpoint, redisEndpoint, dbSecretArn
StorageStack	S3 bucket (solaros-files-{env}) for all PDFs and documents. 12 MFE S3 buckets. 12 CloudFront distributions (one per MFE + one for portal). ACM certificates in ap-south-1 and us-east-1. Route53 A records.	NetworkStack	mfeBucketNames, cfDistributionIds, filesBucketName
QueueStack	10 SQS Standard queues + DLQs, 2 SQS FIFO queues + DLQs, 5 SNS topics with subscriptions, 4 EventBridge rules (payment reminders, follow-up, weekly report, e-way bill expiry), WebSocket API	NetworkStack	queueUrls, topicArns, eventBusArn
ServicesStack	18 Lambda functions (SolarLambda L3 construct each), 5 App Runner services (SolarAppRunner L3 each), IAM roles per service (least-privilege — no DDL, no TRUNCATE), Provisioned Concurrency for billing-service	DataStack + QueueStack + StorageStack	functionArns, appRunnerUrls
ApiGatewayStack	API Gateway HTTP v2, Lambda Authorizer (JWT RS256), WAF WebACL (OWASP), route integrations per service, usage plans per plan tier, custom domain (api.solaros.in + mobile-api.solaros.in)	ServicesStack	apiGatewayUrl, apiGatewayId
MonitoringStack	CloudWatch dashboards per service, P99 latency alarms (threshold 3s), error rate alarms (>1%), DLQ depth alarms (>0 messages), Sentry DSN environment variables, DataDog Lambda layer attachment	ServicesStack + QueueStack	alarmArns, dashboardUrls

SolarLambda L3 Construct (CloudArc extension): One constructor call creates: Lambda function (Node.js 20) + IAM role (Secrets Manager read, SQS send/receive for designated queues, S3 read/write to files bucket only) + optional SQS event source mapping + CloudWatch P99 alarm (threshold 3s) + X-Ray tracing enabled + VPC placement in private subnet. Lambda always receives `rdsProxyEndpoint` — never the direct RDS hostname. This is enforced in the construct constructor, not in service code.

CHAPTER 16

Environment Setup & Local Development

Prerequisites

TOOL	VERSION	INSTALL COMMAND
Node.js	20 LTS	<code>nvm install 20 && nvm use 20</code>
pnpm	9.x	<code>npm install -g pnpm@9</code>
Docker Desktop	Latest	<code>docker.com/desktop</code>
AWS CLI	v2	<code>brew install awscli</code>
AWS CDK CLI	2.x	<code>npm install -g aws-cdk</code>
Python	3.12	<code>pyenv install 3.12</code> (for ai-service only)
Turborepo	Latest	Installed via pnpm workspace — no global install needed

docker-compose Local Services

SERVICE	IMAGE	PORT	PURPOSE
<code>postgres</code>	<code>pgvector/pgvector:pg16</code>	5432	Primary DB with pgvector extension. Seed creates public schema + demo tenant schema with sample data.
<code>redis</code>	<code>redis:7-alpine</code>	6379	Sessions, OTP cache (10-min TTL), GST token cache, rate limiting, stock pub/sub
<code>localstack</code>	<code>localstack/localstack:latest</code>	4566	Emulates: S3, SQS, SNS, Secrets Manager, Lambda (integration tests), CloudWatch
<code>mailhog</code>	<code>mailhog/mailhog</code>	8025	Local SMTP — catches all SES emails in dev. View at <code>http://localhost:8025</code>
<code>clickhouse</code>	<code>clickhouse/clickhouse-server:23</code>	8123	Local analytics DB — only for reporting-service development

Required Environment Variables

VARIABLE	LOCAL DEV VALUE	PROD SOURCE
DATABASE_URL	postgresql://solaros:secret@localhost:5432/solaros_dev	RDS Proxy endpoint via Secrets Manager
REDIS_URL	redis://localhost:6379	ElastiCache endpoint via Secrets Manager
AWS_ENDPOINT_URL	http://localhost:4566	Not set in prod (uses real AWS endpoints)
AWS_REGION	ap-south-1	ap-south-1
JWT_PRIVATE_KEY_PATH	./keys/private.pem	Secrets Manager (auto-rotated annually)
JWT_EXPIRY	900 (15 minutes)	900
S3_BUCKET_FILES	solaros-files-dev	solaros-files-prod
SQS_NOTIFICATION_QUEUE_URL	LocalStack SQS URL	Real SQS URL from QueueStack output
SQS_INVOICE_QUEUE_URL	LocalStack FIFO SQS URL	Real FIFO SQS URL from QueueStack output
RAZORPAY_KEY_ID / RAZORPAY_WEBHOOK_SECRET	rzp_test_xxx / test_secret	Secrets Manager
WA_PHONE_NUMBER_ID / WA_ACCESS_TOKEN	Mock — skip WhatsApp sends in dev	Secrets Manager (token auto-rotated)
GST_ASP_BASE_URL / GST_CLIENT_ID / GST_CLIENT_SECRET	GSP sandbox endpoint + credentials	Secrets Manager
ANTHROPIC_API_KEY	sk-ant-xxx	Secrets Manager
FCM_SERVER_KEY	Firebase test project key	Secrets Manager
SENTRY_DSN	Dev Sentry project URL	Prod Sentry project URL

CHAPTER 17

API Contract Conventions

Response Envelope

Every response uses the same envelope: **data** (payload or null), **error** (error object or null), **meta** (page, limit, total, request_id, timestamp). On success: data has payload, error is null. On failure: data is null, error has **code**, **message**, and optional **field** for validation errors.

Pagination

All list endpoints: `?page=1&limit=20&sort=created_at:desc&filter[status]=Active`
Meta always returns page, limit, total, has_next_page.
Cursor-based pagination available for high-volume tables (serial_numbers — up to 1M rows per tenant).

Optimistic Locking

All PUT/PATCH requests must include **version** field matching current DB version. If mismatch → 409 Conflict with current version in response. Client re-fetches and retries. Prevents concurrent overwrites on quotations, sales orders, and customer records.

API Versioning

All routes: `/v1/*`. Breaking changes introduce `/v2/*`. v1 maintained 12 months post v2 launch. Deprecation announced via **Deprecation: true** and **Sunset: YYYY-MM-DD** response headers. Mobile GraphQL uses schema **@deprecated** directives.

ID Format

All internal PKs: UUID v4. Display IDs (SO-2026-000001, ENQ-2026-000001) generated in service layer using number series config from **tenant.config**. Display IDs for UI only — API calls always use UUIDs as identifiers.

Idempotency

All POST mutations accept optional **Idempotency-Key** header (UUID). If same key seen within 24h (Redis TTL), original response returned without re-executing. Critical for E-Invoice generation — prevents duplicate GSTN submissions on network retry.

Standard Error Codes

HTTP	ERROR CODE	WHEN IT FIRES
400	VALIDATION_ERROR	Zod schema validation failed. field identifies the specific failing field.
401	UNAUTHORIZED	Missing, expired, or invalid JWT
403	FORBIDDEN	JWT valid but role lacks permission for this action on this resource
404	NOT_FOUND	Resource not found in tenant schema (RLS may silently return 0 rows)
409	VERSION_CONFLICT	Optimistic lock version mismatch on PUT/PATCH — client must re-fetch
409	DUPLICATE_ENTITY	Phone, email, GSTIN, or SKU already exists in this tenant
422	BUSINESS_RULE_VIOLATION	Business logic blocked: cannot delete customer with open orders; quotation below floor needs approval; SO cannot dispatch without IRN
429	RATE_LIMIT_EXCEEDED	API Gateway usage plan limit hit for this plan tier
503	GST_API_UNAVAILABLE	GSTN IRP or ASP/GSP returned 5xx. Retry-After header set. Dispatch team notified via WhatsApp.
503	WHATSAPP_API_UNAVAILABLE	Meta Cloud API returned 5xx. Message queued for retry via DLQ.

CHAPTER 18

Day 1 Engineering Checklist

Follow in order. By end of Day 1: local dev stack running, staging AWS environment provisioned, auth-service live on App Runner staging, and one lead created end-to-end via API call.

HOURS 0–2 · REPO SETUP

- Create GitHub org: techpotato-software
- Create repo: solaros (private)
- Enable branch protection on main
- Require 1 PR review before merge
- Add CODEOWNERS file
- pnpm init + pnpm-workspace.yaml
- Add turbo.json pipeline config
- Add .nvmrc → node 20
- Invite all engineers with write access

HOURS 2–4 · PACKAGES + CLOUDARC

- Copy CloudArc → packages/infra
- Rename constructs: SolarLambda, SolarAppRunner, MfeCdn
- Create packages/types with all entity interfaces
- Create packages/db with full Prisma schema
- Create packages/utis: gst.ts, solar.ts, validators.ts, currency.ts
- Create packages/config: Zod env parsing — fail fast on startup
- Create packages/queue: SQS publisher + consumer helpers
- Create packages/auth-client: JWT verify + tenant middleware

HOURS 4–6 · LOCAL DEV STACK

- Write docker-compose.yml with all 5 services
- docker-compose up -d → verify all containers healthy
- prisma migrate dev → public schema created in postgres
- pnpm db:seed → demo tenant + admin user created
- Generate RS256 key pair → store in /keys (gitignored)
- cp .env.example .env.local → fill all vars from table above
- pnpm dev → verify all services start locally
- Open localhost:5173 → shell MFE loads

HOURS 6–8 · AUTH SERVICE

- Scaffold auth-service (Fastify + CSR folder structure)
- Implement POST /oauth/register → user + tenant + schema
- Implement POST /oauth/token → JWT access (15min) + refresh (7d)
- Implement GET /.well-known/jwks.json → public key
- Implement tenant middleware → search_path set per request
- Test: register → login → decode JWT → verify tenantId correct
- Test RLS: user A cannot read leads from tenant B

HOURS 8–10 · CDK + FIRST LEAD

- cdk bootstrap → AWS account ap-south-1
- Write NetworkStack → cdk deploy staging
- Write DataStack → RDS + Proxy + Redis → deploy
- Scaffold leads-service (Hono + CSR)
- POST /v1/leads → Zod validate + create lead
- GET /v1/leads → list with pagination + RLS
- Deploy auth + leads to staging (App Runner + Lambda)
- End-to-end: JWT → POST /v1/leads → verify in RDS
- Set up .github/workflows/ci.yml → CI passes green
- Send team update: staging URL + repo link + README

Week 1 Sprint Goals

DAY	GOAL	SERVICES / PACKAGES COMPLETED
Day 1	Repo setup, auth-service live on staging, first lead created end-to-end via API	auth-service, packages/types, packages/db, CloudArc NetworkStack + DataStack
Day 2	Customers + Leads services. Shell MFE scaffold. QueueStack deployed. SQS integration tested locally.	customers-service, leads-service, shell-suite scaffold, QueueStack CDK
Day 3	Enquiry + Quotation services. MFE Leads + Customers UI with real API calls. Floor-price approval flow tested.	enquiry-service, quotation-service, mfe-leads, mfe-customers, packages/ui base components
Day 4	Sales Order service. Payment milestone tracking. WhatsApp Meta account setup + WABA verification started.	sales-order-service, notification-service skeleton, whatsapp-service skeleton, mfe-sales
Day 5	Full staging demo with Oriana team. Lead → Enquiry → Quotation → SO flow tested end-to-end. Playwright integration tests.	All services above deployed to staging. Playwright smoke tests passing. Oriana Phase 1 sign-off.

✔ Definition of Done — Day 1

- GitHub repo live with branch protection and all engineers added as collaborators
- Local dev stack starts cleanly with `docker-compose up -d && pnpm install && pnpm dev` — no manual steps
- Full Prisma schema migrated to local PostgreSQL with seed data loaded
- auth-service responds at `localhost:3001/health` with 200 OK
- POST `/v1/leads` on staging returns 201 with leadId in under 500ms p99
- RLS verified: User from tenant A cannot read leads from tenant B (test in CI)
- CI pipeline passes green on first PR merge to main (lint + typecheck + test)
- All engineers have AWS staging console access (least-privilege IAM role)
- README documents complete local onboarding in under 30 minutes for a new hire

CHAPTER 19

Zoho Books GST Integration — Phase 1 Strategy

Rather than pursuing direct GSTN ASP/GSP registration in Phase 1 (which takes 4–6 weeks and requires significant compliance overhead), SolarOS will integrate with **Zoho Books** as the GST compliance layer. Zoho Books is already a certified GSTN ASP and handles E-Invoice (IRN generation) and E-way Bill natively. SolarOS acts as the source of truth for all business data and delegates GST document generation to Zoho Books via its REST API + webhook callbacks.

Phase 1 vs Phase 2 GST Strategy

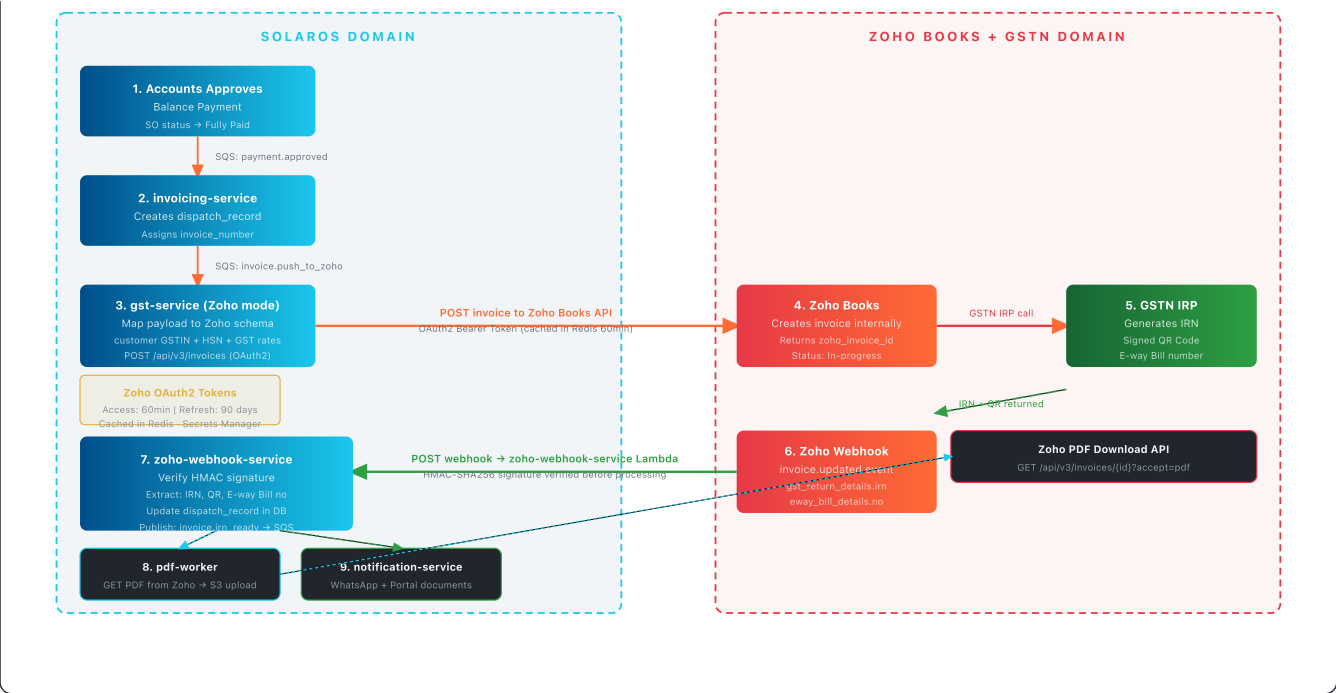
Phase 1 (Months 1–8): Zoho Books integration via REST API + webhooks. SolarOS pushes invoice data to Zoho Books → Zoho generates IRN + E-way Bill → webhooks deliver results back → SolarOS stores IRN and serves documents. No GSTN registration needed from SolarOS side.

Phase 2 (Months 9+): Replace Zoho Books with direct GSTN ASP/GSP integration (own gst-service). Zero code changes in invoicing-service — only the gst-service implementation swaps. All stored IRNs, E-way Bills remain valid.

How Zoho Books GST Integration Works

STEP	ACTOR	ACTION	TECHNICAL DETAIL
1	SolarOS invoicing-service	Balance payment approved by accounts. Invoice record created in SolarOS DB.	SO status → Fully Paid. dispatch_record row created with invoice_number.
2	SolarOS gst-service (Zoho mode)	Map SolarOS dispatch_record to Zoho Books invoice payload. POST to Zoho Books Invoices API.	<code>POST /api/v3/invoices</code> with OAuth2 bearer token. Payload maps customer GSTIN, line items, HSN codes, GST rates.
3	Zoho Books	Creates invoice internally. Pushes to GSTN IRP to generate IRN. Returns Zoho invoice_id.	Zoho Books is a certified GSTN ASP — handles all IRP communication natively. IRN generated within 3–30 seconds.
4	Zoho Books → SolarOS webhook	Zoho sends webhook to SolarOS <code>zoho-webhook-receiver</code> Lambda when invoice is GST-verified (IRN ready).	Webhook event: <code>invoice.updated</code> with <code>gst_return_details.irn</code> and <code>eway_bill_details.eway_bill_no</code> .
5	SolarOS zoho-webhook-service	Verifies Zoho webhook HMAC signature. Extracts IRN, QR code, E-way Bill number from payload.	Stores in <code>dispatch_records.e_invoice_irn</code> and <code>dispatch_records.eway_bill_number</code> .
6	SolarOS pdf-worker	Fetches signed invoice PDF from Zoho Books PDF download API. Uploads to S3.	<code>GET /api/v3/invoices/{id}?accept=pdf</code> . PDF stored at <code>s3://solaros-files/invoices/{dispatch_id}.pdf</code> .
7	SolarOS notification-service	Sends dispatch documents to customer via WhatsApp + stores in portal.	WhatsApp template <code>dispatch_initiated</code> with presigned PDF links. Portal documents updated.

 Zoho Books GST Integration — Detailed Flow



Zoho Books API Mapping — SolarOS to Zoho Schema

SOLAROS FIELD	ZOHO BOOKS FIELD	NOTES
<code>dispatch_records.invoice_number</code>	<code>invoice_number</code>	SolarOS-generated sequential number used as Zoho invoice number
<code>customers.gstin</code>	<code>gst_no</code> (contact)	Customer GST number — contact must exist in Zoho Books or created on-the-fly
<code>customers.business_address</code>	<code>billing_address</code>	State code extracted for CGST/IGST determination
<code>sales_orders.line_items[].product_id</code>	<code>line_items[].item_id</code>	Products synced to Zoho Books item catalog on first use (one-time setup)
<code>inventory_items.hsn_code</code>	<code>line_items[].hsn_or_sac</code>	Critical for E-Invoice. 8541 for solar panels.
<code>inventory_items.gst_rate_pct</code>	<code>line_items[].tax_id</code>	Tax rates pre-configured in Zoho Books. Mapped by GST % value.
<code>quotations.grand_total</code>	<code>total</code>	Cross-validated against Zoho computed total before sending
<code>tenants.gstin</code>	Zoho Organization GSTIN	Set at Zoho Books organization level — one org per SolarOS tenant

Zoho Webhook Event Payload Structure

WEBHOOK FIELD PATH	MAPS TO SOLAROS	REQUIRED
<code>data.invoice.invoice_id</code>	Stored in <code>dispatch_records.zoho_invoice_id</code>	Yes — used for PDF download

WEBHOOK FIELD PATH	MAPS TO SOLAROS	REQUIRED
data.invoice.status	Must be <code>sent</code> or <code>overdue</code> to trigger IRN extract	Yes
data.invoice.gst_return_details.irn	dispatch_records.e_invoice_irn	Yes — stored permanently
data.invoice.gst_return_details.qr_code	dispatch_records.e_invoice_qr_code	Yes — printed on invoice
data.invoice.eway_bill_details.eway_bill_no	dispatch_records.eway_bill_number	Yes
data.invoice.eway_bill_details.expiry_date	dispatch_records.eway_bill_expiry	Yes — EventBridge alert 24h before

Phase 2 Migration — Zoho to Own GSTN API

ASPECT	PHASE 1 (ZOH0)	PHASE 2 (OWN GSTN API)	MIGRATION EFFORT
Code Change	gst-service implements Zoho Books client	gst-service implements GSTN ASP/GSP client	Only gst-service changes. invoicing-service untouched.
Stored Data	IRN, E-way Bills stored in SolarOS DB	Same DB schema. New IRNs from new source.	Zero migration. Old IRNs stay valid.
SQS Events	Same event names: invoice.irn_ready	Identical event names	Zero downstream changes
Cost	Zoho Books subscription (~₹5,000–15,000/month per org)	GSTN API cost (~₹0.50–2 per invoice)	Significant cost reduction at scale
Speed	IRN in 5–60 seconds (Zoho queue)	IRN in 2–10 seconds (direct)	Better customer experience
Control	Zoho availability dependency	Full control	Higher reliability

New Service Added: zoho-webhook-service — A new Lambda that receives POST requests from Zoho Books on invoice status changes. It is the only entry point for Zoho data into SolarOS. It verifies the Zoho webhook HMAC-SHA256 signature using the shared secret stored in Secrets Manager, then publishes `invoice.irn_ready` to SQS. The existing `gst-service` Lambda handles the Zoho Books REST API calls (sending invoices). Both services are replaced in Phase 2 with GSTN direct integration — zero other changes needed.

New DB Fields for Zoho Integration

TABLE	NEW FIELD	TYPE	PURPOSE
dispatch_records	zoho_invoice_id	TEXT	Zoho Books invoice ID — used for PDF download + status polling
dispatch_records	zoho_sync_status	TEXT	pending synced irn_received failed — tracks Zoho sync state

TABLE	NEW FIELD	TYPE	PURPOSE
dispatch_records	zoho_sync_error	TEXT	Last error message if sync failed — for retry UI
tenants	zoho_org_id	TEXT	Zoho Books organization ID for this tenant
tenants	zoho_access_token	TEXT (encrypted)	Cached OAuth2 access token — refreshed automatically
tenants	gst_integration_mode	TEXT	zoho direct — switches Phase 1 vs Phase 2 behavior in gst-service

CHAPTER 20

WhatsApp Automation Platform — Cross-Industry Standalone Vertical

India has 500M+ WhatsApp users. Over 90% of Indian SMSMEs communicate with customers, suppliers, and teams entirely on WhatsApp. **SolarOS WhatsApp Automation** is a standalone product that can be sold independently — not just to solar companies, but to any MSME in any industry that runs its operations on WhatsApp.

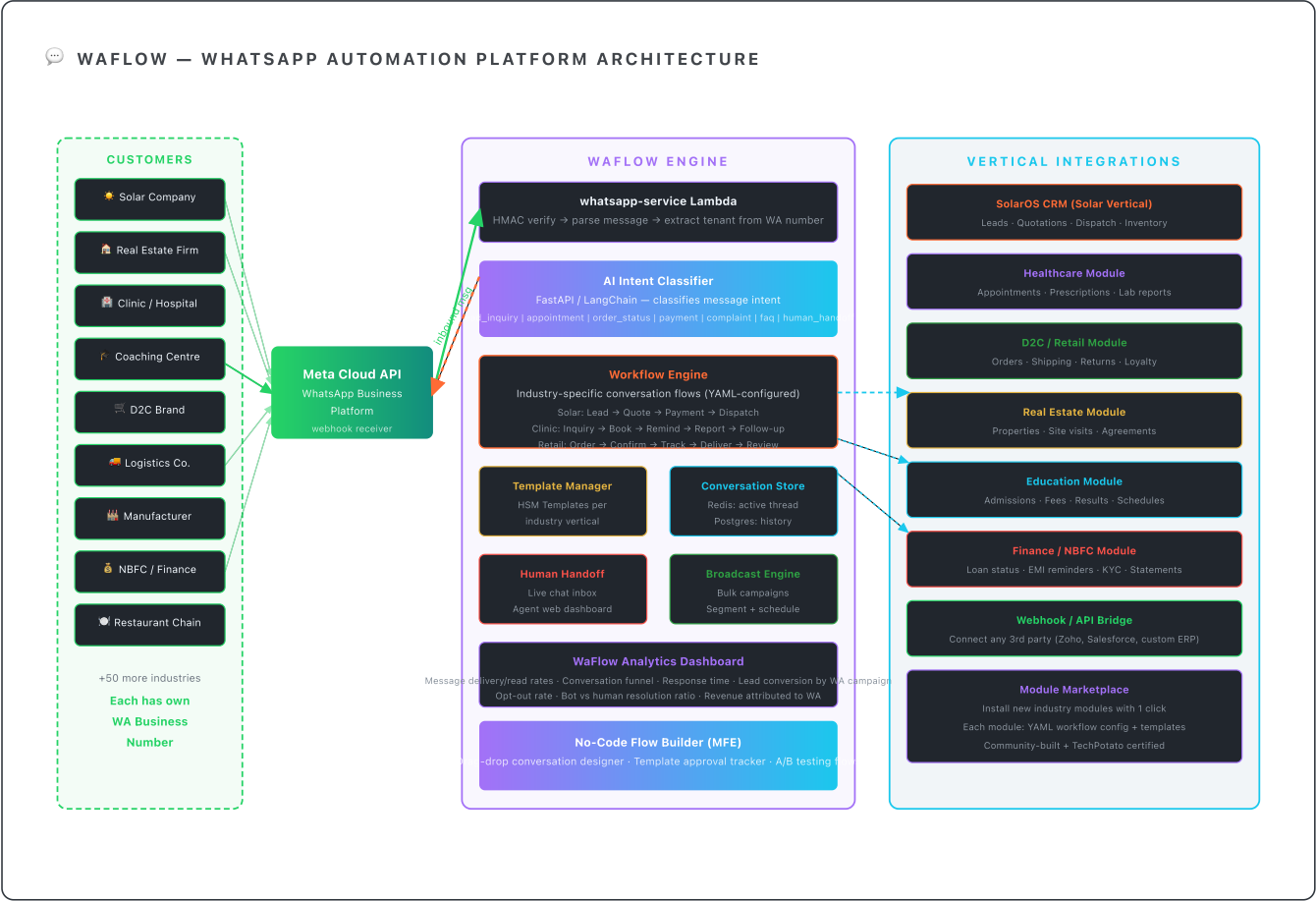
The WhatsApp vertical is packaged as "**WaFlow**" — a no-code/low-code WhatsApp business automation platform. It can be bundled with SolarOS or sold as a standalone SaaS product to non-solar businesses.

Market Opportunity: 63 million registered MSMEs in India. Less than 5% use any CRM. 95% use WhatsApp for business. WaFlow targets the 60M businesses that are "WhatsApp-native" and have no software — starting at ₹999/month with no installation required. Every business that signs up becomes a potential upsell to the full SolarOS platform.

Industry Verticals — WaFlow Use Cases

INDUSTRY	WHATSAPP AUTOMATION USE CASES	KEY WORKFLOWS
🌞 Solar Energy	Lead capture, quotation sharing, payment reminders, dispatch tracking, serial traceability, portal access	Lead → Quote → Payment → Dispatch → Delivery confirmation
🏠 Real Estate	Property inquiry auto-reply, virtual tour booking, site visit scheduling, document collection, payment reminders	Inquiry → Site visit → Agreement → Registration → Handover
🏥 Healthcare / Clinics	Appointment booking, reminder notifications, prescription sharing, lab report delivery, follow-up reminders	Appointment → Reminder → Consultation → Report → Follow-up
🎓 Education / Coaching	Admission inquiry auto-reply, fee reminder, study material sharing, exam schedule, result notification	Inquiry → Demo → Enrollment → Fee → Material → Result
🛒 Retail / D2C	Order confirmation, shipping tracking, return processing, loyalty rewards, flash sale broadcasts	Order → Confirmation → Shipping → Delivery → Review request
🚚 Logistics / Transport	Shipment pickup confirmation, real-time tracking updates, delivery proof, POD collection, driver assignment	Booking → Pickup → In-transit → Delivery → POD

INDUSTRY	WHATSAPP AUTOMATION USE CASES	KEY WORKFLOWS
 Manufacturing / B2B	PO acknowledgment, production status, dispatch intimation, invoice sharing, GST document delivery	PO → Production → Dispatch → Invoice → Payment
 Finance / Lending	Loan application status, EMI reminders, KYC document collection, statement delivery, payment confirmation	Application → KYC → Approval → Disbursal → EMI reminders
 Restaurants / F&B	Table booking, order confirmation, delivery tracking, feedback collection, loyalty program, menu sharing	Booking → Confirmation → Reminder → Feedback



WaFlow Technical Architecture — Key Components

COMPONENT	TECHNOLOGY	PURPOSE
Webhook Receiver	Hono Lambda (PCCU=2)	Receives all inbound messages from Meta Cloud API. HMAC-SHA256 verified. Routes to correct tenant via WhatsApp Business Number lookup in Redis.
Intent Classifier	FastAPI + LangChain + OpenAI/Claude	Classifies inbound message intent into predefined categories. Industry-specific classification models. Confidence score determines bot vs human handoff.
Workflow Engine	State machine (XState) + Redis	Each conversation is a state machine. States persisted in Redis (active) and PostgreSQL (history). YAML-configured flows per industry module.
Template Manager	Node.js Lambda + PostgreSQL	Manages WhatsApp HSM template library per tenant per industry. Tracks Meta approval status. A/B testing different template variants.

Component	Technology	Purpose
Broadcast Engine	SQS FIFO + Lambda (rate-limited)	Bulk campaign sender. Respects Meta rate limits (1000 messages/sec per WABA). Segment filtering. Schedule support. Opt-out enforcement.
Human Handoff Inbox	WebSocket API + React MFE	Real-time agent inbox. Assigns conversations to human agents when bot confidence <70% or customer requests human. SLA timer per conversation.
No-Code Flow Builder	React MFE + dnd-kit	Drag-drop visual conversation designer. Node types: Message, Condition, API Call, Delay, Tag Contact, Assign Agent. YAML exported and stored per tenant.
Conversation Database	PostgreSQL + Redis	Full conversation history per contact per tenant. Redis for active thread state (24h TTL). PostgreSQL for permanent history and analytics.

WaFlow Pricing Model (Standalone Product)

Plan	Price	Conversations/Month	Features	Target
Starter	₹999/month	500	1 WA number, 5 templates, basic chatbot, lead capture	Solo entrepreneur, small clinic
Growth	₹2,999/month	2,000	1 WA number, 20 templates, AI intent, broadcast, human inbox	D2C brand, coaching centre
Business	₹7,999/month	10,000	3 WA numbers, unlimited templates, API access, 5 agents, analytics	Multi-branch clinic, real estate agency
Enterprise	₹24,999/month	Unlimited	10+ WA numbers, custom flows, white-label, priority support, SLA	Large NBFC, national retail chain
Overage	₹0.50/conversation	Beyond plan limit	Auto top-up available	All plans

CHAPTER 21

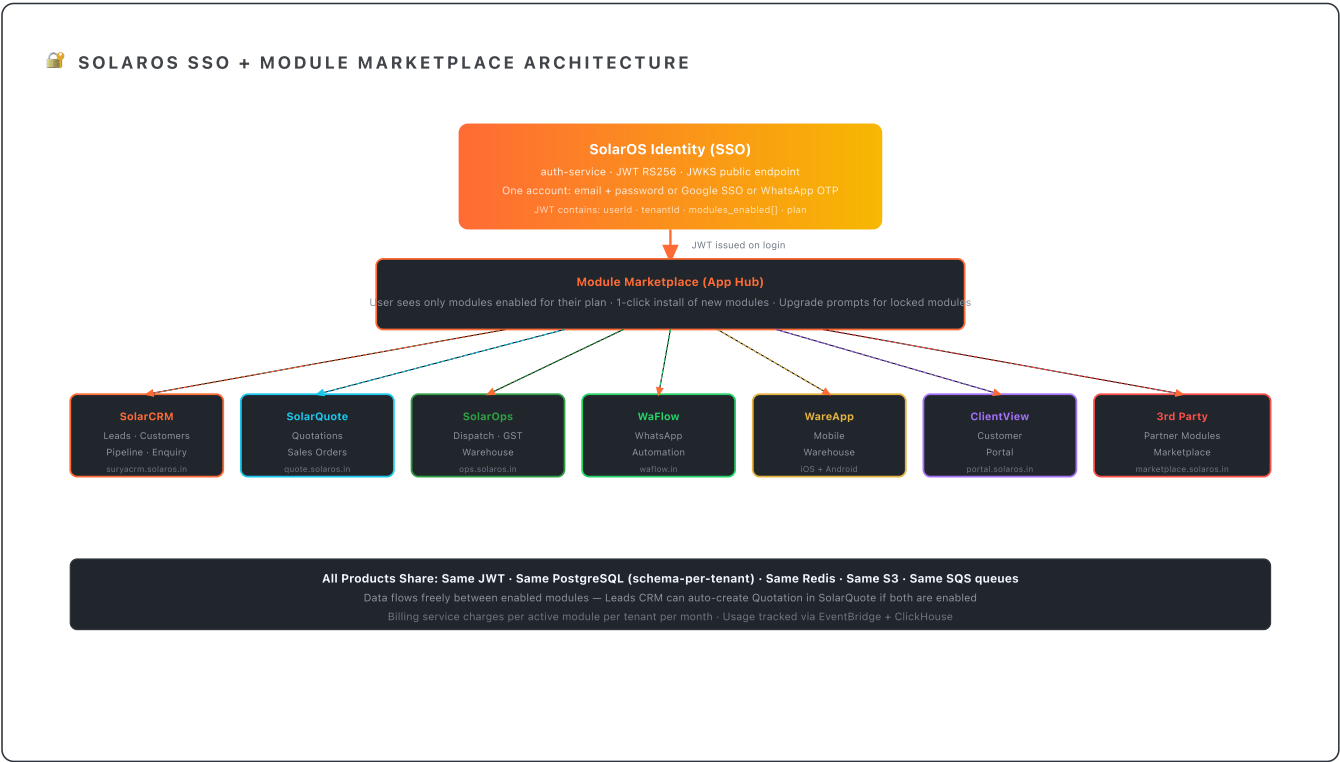
Module Marketplace & SSO Architecture — Individual Module Sales

SolarOS is architected for **modular monetization** from day one. Every module (Leads, Quotations, Warehouse, WhatsApp, Mobile App) can be sold as a standalone product with its own pricing, its own login, and its own tenant. A customer can start with just "Leads CRM" for ₹999/month and add "Quotation Manager" when they're ready — without changing their login or losing data.

The SSO model mirrors Zoho's approach: one SolarOS account, multiple products. Customers choose their modules like a marketplace. Partners and resellers can bundle modules for their specific industry and sell as white-labeled products.

MODULE	STANDALONE PRODUCT NAME	PRICE/MONTH	TARGET CUSTOMER
Leads + Customers + Pipeline	SolarCRM Lite	₹999	Solo consultants, small resellers
+ Quotations + Sales Orders	SolarCRM Pro	₹2,999	Growing distributors
+ Dispatch + GST (Zoho)	SolarOps	₹5,999	EPC companies with dispatch team
+ Warehouse + Purchase	SolarOps+	₹9,999	Distributors with warehouse
All modules + Mobile Apps	SolarOS Enterprise	₹24,999	Large EPC, national distributors
WhatsApp Automation only	WaFlow	₹999–24,999	Any MSME — not solar-specific
Mobile Warehouse App only	WareApp	₹499/user/month	Standalone warehouse operations
Customer Portal only	ClientView	₹1,999	Any B2B business needing customer portal

SSO Architecture — One Login, All Products



Module Entitlement System

MECHANISM	IMPLEMENTATION	DETAIL
JWT Entitlements	modules_enabled: ["leads","quotations","dispatch"] in JWT payload	auth-service reads tenants.modules_enabled from DB and embeds in JWT. MFE shell shows/hides navigation based on this array. API Gateway blocks routes for disabled modules.

MECHANISM	IMPLEMENTATION	DETAIL
Feature Flags	<code>public.feature_flags</code> table + Redis cache	Per-feature toggles beyond module level. Used for gradual rollouts and A/B testing. Checked in service layer, not gateway.
Module Activation	Billing service on Razorpay subscription webhook	When customer upgrades plan or adds module: billing-service updates <code>tenants.modules_enabled</code> → auth-service invalidates JWT cache → next login includes new module.
Trial Mode	14-day trial per module with usage cap	Module added to JWT with trial flag. Usage counted via EventBridge. Trial expiry → module locked but data preserved. Upgrade prompt shown in UI.
White-Label	Custom domain per tenant per module	Enterprise plan allows <code>crm.clientname.com</code> → CloudFront → same MFE. Logo, colors from <code>tenants.config.branding</code> . SSO works cross custom domains via JWT.

CHAPTER 22

Deep-Dive Architecture Diagrams

This chapter provides additional detailed architecture views for the most complex modules. Each diagram deep-dives into data flows, service interactions, and state transitions that cannot be shown in the high-level architecture overview.

Diagram A — Event-Driven Architecture: Complete Queue Map & Data Flow

📄 EVENT-DRIVEN ARCHITECTURE — COMPLETE DATA FLOW WITH ALL QUEUES, CONSUMERS & FANOUT



Every service also publishes to: audit-queue (all mutations) + notification-queue (when user action requires notification)
No service calls another service synchronously via internal API — all cross-service communication is via SQS
Exception: auth-service (JWT verification via JWKS) and gst-service (called by invoicing-service via SQS, not HTTP)

Diagram B — Complete Lead-to-Cash Data Flow

LEAD-TO-CASH — COMPLETE DATA FLOW ACROSS ALL MODULES

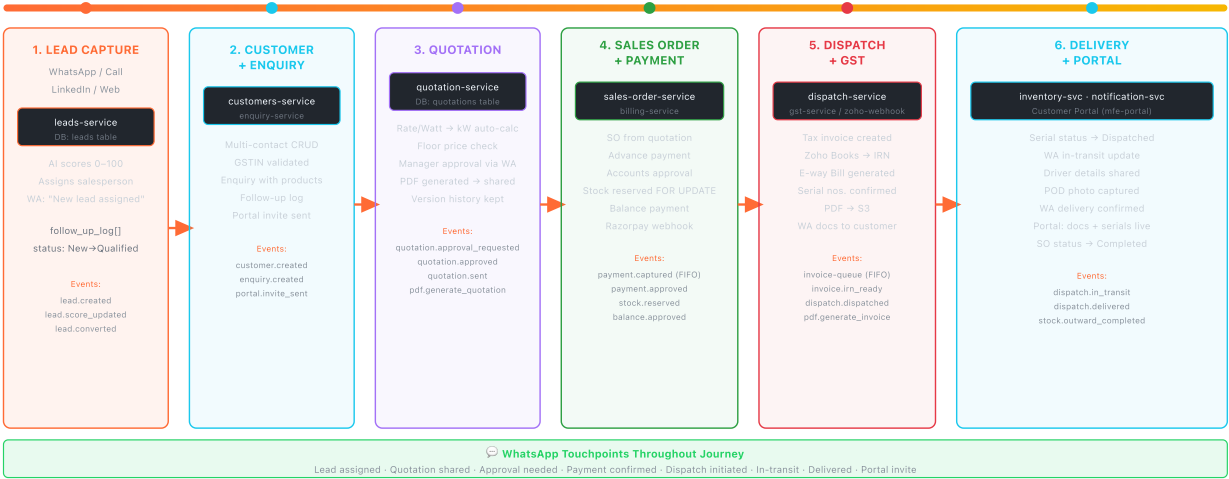
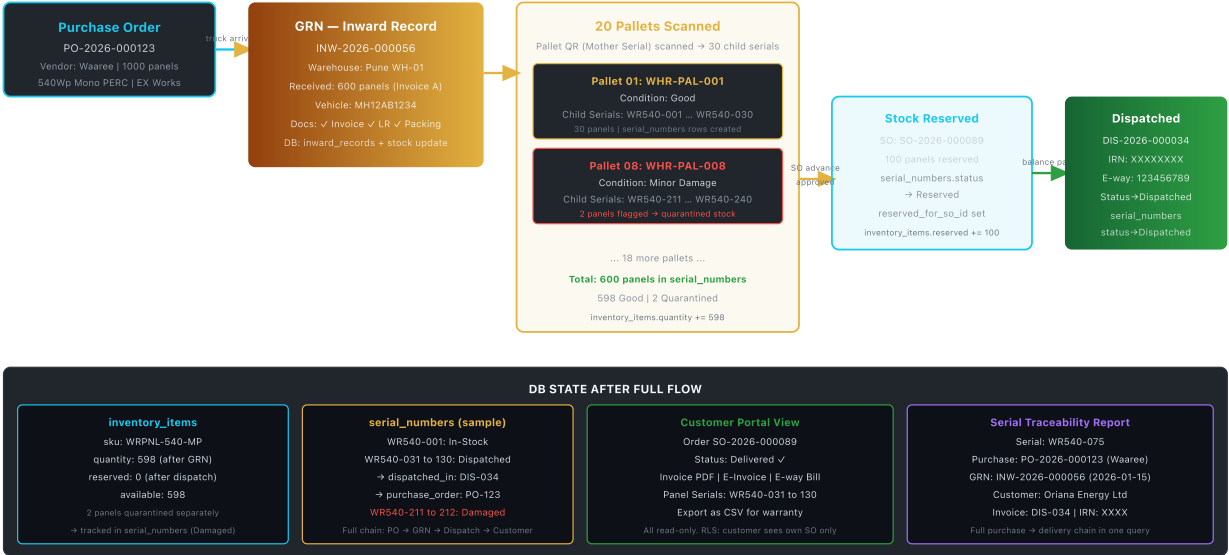


Diagram C — Warehouse Serial Tracking: Pallet → Panel → Customer Traceability

WAREHOUSE SERIAL TRACEABILITY — PALLET TO PANEL TO CUSTOMER COMPLETE FLOW



Technical Initiation Document v1.0 · May 2026 · Confidential · For Internal Engineering Team Only

TechPotato Software LLP · techpotato.in · Built with ♥ for Indian Solar MSMEs

18 Microservices

12 MFEs

13 PRD Modules

India-Native GST

WhatsApp-First

AWS Serverless

React Native Mobile

Zero-Trust Security